Coding Assignment 4: The Calculator Learns to Rewrite

Auqib Hamid Lone
Department of Computer Science & Engineering, GCET Ganderbal

September 9, 2026

Due: two weeks from issue. There is no automatic late penalty — if you need more time, ask before the deadline and an extension will be granted (see the course policy in the syllabus).

4.1 Why this problem

Week 3 gave the calculator two finished stages: it could check brackets and *evaluate* a postfix string it was handed. But nobody types postfix — humans write infix, with precedence, associativity, and parentheses mixed together, and something has to rewrite it first. That rewrite is the missing middle stage of the pipeline, and it is also the first program in this course where the *order of equal-precedence operators* silently changes the answer: pop a waiting `^` too eagerly and `x^y^z` computes `(x^y)^z` instead of `x^(y^z)`. This assignment is that rewrite stage, twice over: Lab P3 (infix to postfix and prefix) and Lab P4 (prefix back to postfix), all running on the Week 3 stack discipline of one left-to-right pass with each token pushed and popped at most once.

4.2 Your task

Implement the three expression conversions in C as **pure single-pass functions over caller-provided buffers with an explicit status code on every fallible path**. The contract is given to you in `expr.h` — *do not modify it.* Copy `starter_expr.c` to `expr.c` and fill in all three functions:

```
ExprStatus infix_to_postfix(const char *infix, char *postfix, size_t out_size);
ExprStatus infix_to_prefix(const char *infix, char *prefix, size_t out_size);
ExprStatus prefix_to_postfix(const char *prefix, char *postfix, size_t out_size);
```

P3a — infix to postfix. The lecture's shunting-yard scan: operands append to output; `(` pushes; `)` pops to output until `(` (both parens discarded); operator `op` pops waiting operators of `>=` precedence (strictly `>` if `op` is right-associative), then pushes `op`; at the end, pop everything to output. Precedence: `^` (highest, right-associative) > `* /` (medium, left) > `+ -` (low, left).

P3b — infix to prefix. The lecture's reversal route: reverse the *token* stream (not the characters — `12` must stay `12`), swap `(` with `)`, run the postfix pass under the mirror rule (pop only on *strictly greater* precedence), then reverse the result. The paren swap is load-bearing; without it every bracket binds backwards.

P4 — prefix to postfix. Scan *right to left* with a stack of strings: operand pushes as a string; operator `op` pops `a` first and `b` second, then pushes `a + " " + b + " " + op`. Direction matters:

right-to-left meets operands before the operator that combines them. A single operand alone is the identity (`"a"` → `"a"`).

Tokens and output format. Operands are single letters (`a`–`z`) *or* non-negative multi-digit integers (`12`, `305`); operators are `+ - * / ^`; infix alone may use `( )`. Spaces and tabs may appear anywhere and are ignored, except that they delimit multi-digit numbers (`"12+34*5"` and `"12 + 34 * 5"` are the same expression). There is no unary minus in infix: a leading `-` is always binary subtraction, so `"a+-b"` is invalid. Every output is space-separated tokens with exactly one space, no trailing space (e.g. `"x y z * + w -"`).

Status codes. Read all six `ExprStatus` codes in `expr.h` before coding — each error maps to exactly one of them. On *any* non-`OK` status the output buffer is unspecified and must not be read.

4.3 Constraints

- Implementation must be the lecture's one-stack passes — no expression trees, no recursive-descent parser, no `system` calls, no heap allocation required (fixed-size local stacks suffice), no global/static mutable state, no `strtok` shortcuts that hide the token scan.
- Error paths use the explicit status codes — a submission that returns `EXPR_OK` with a wrong string, or the wrong code on an error input, fails the hidden suite's status traps (check *both* the return value and the buffer in your own testing).
- Correctness is graded against the contract; your code must compile with `gcc -Wall -Wextra` (or MSVC `/W4` on Windows) without errors or warnings.
- Complexity: each conversion makes one pass at $O(n)$ time worst case in the input length, holding at most $O(n)$ auxiliary space for n tokens (each token pushed and popped at most once).

4.4 Worked examples (these are the visible tests)

Example 1 — P3a, infix to postfix. `infix_to_postfix("x+y*z-w")` returns `EXPR_OK` with `"x y z * + w -"`. `infix_to_postfix("x^y^z+w")` returns `EXPR_OK` with `"x y z ^ ^ w +"` (the second `^` must *not* pop the first — right-associativity). `infix_to_postfix("(x+y)*(z-w)")` returns `EXPR_OK` with `"x y + z w - *"`.

Example 2 — P3b and multi-digit operands. `infix_to_prefix("x+y*z-w")` returns `EXPR_OK` with `"- + x * y z w"`. `infix_to_postfix("12+34*5")` returns `EXPR_OK` with `"12 34 5 * +"` (the numbers stay whole). `infix_to_prefix("12+34*5")` returns `EXPR_OK` with `"+ 12 * 34 5"`.

Example 3 — P4, prefix to postfix. `prefix_to_postfix("*+ab+cd")` returns `EXPR_OK` with `"a b + c d + *"`. `prefix_to_postfix("+ 12 * 34 5")` returns `EXPR_OK` with `"12 34 5 * +"`. `prefix_to_postfix("a")` returns `EXPR_OK` with `"a"`.

4.5 What to submit

1. `expr.c` — your completed implementation (edit `starter_expr.c`).
2. `expr.h` — unchanged, but include it so the submission compiles standalone.

4.6 Getting started

1. Copy `starter_expr.c` to `expr.c` and fill in the three function bodies.

2. Sanity-check locally:

   ```
   gcc -Wall -Wextra -o test_visible test_visible.c expr.c
   ./test_visible
   ```

   All three worked examples should pass. (On Windows with MSVC, see the build lines at the top of `starter_expr.c`.)

3. Submit `expr.c` and `expr.h`. The autograder compiles your `expr.c` against a *hidden* test suite and reports pass/fail. The hidden suite covers: `NULL`/empty/whitespace inputs; mismatched parens in both directions; right-associative `^` chains vs. left-associative `-` and `/` chains; multi-digit operands with and without spaces; invalid tokens and malformed infix grammar (empty parens, doubled operators, doubled operands, missing operators, unknown characters, rejected unary minus); invalid prefix (too-few-operands, leftover operands, bad tokens) and the single-operand identity; output buffers too small; and seeded random batches of expression trees whose fully-parenthesised infix is checked against post-order/pre-order ground truth, including a prefix round-trip.